

## Q) Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

### 1. Aim

To design and implement a **Bank Account Management System** using Java classes and objects to perform banking operations such as deposit, withdrawal, interest calculation, and fund transfer.

### 2.Steps

1. Create a base class **Account** with common attributes.
2. Create a subclass **SavingsAccount** with an interest rate.
3. Create a subclass **CurrentAccount** with an overdraft limit.
- 4.Implement methods for:
  - Deposit
  - Withdraw
  - Calculate Interest
  - Transfer Amount
- 5.Create objects for different account holders.
- 6.Perform transactions and display updated balances.

### 3.Algorithm

#### Algorithm: Bank Account Management System

1. Start
2. Create class Account with:

- accountNumber
  - balance
  - accountType
3. Create methods:
    - deposit()
    - withdraw()
    - transfer()
    - display()
  4. Create class SavingsAccount inheriting Account
    - Add interestRate
    - Create calculateInterest()
  5. Create class CurrentAccount inheriting Account
    - Add overdraftLimit
  6. Create objects:
    - Savings Account object
    - Current Account object
  7. Deposit amount into accounts.
  8. Withdraw amount from accounts.
  9. Calculate interest for savings account.
  10. Transfer money between accounts.
  11. Display account details.
  12. Stop.

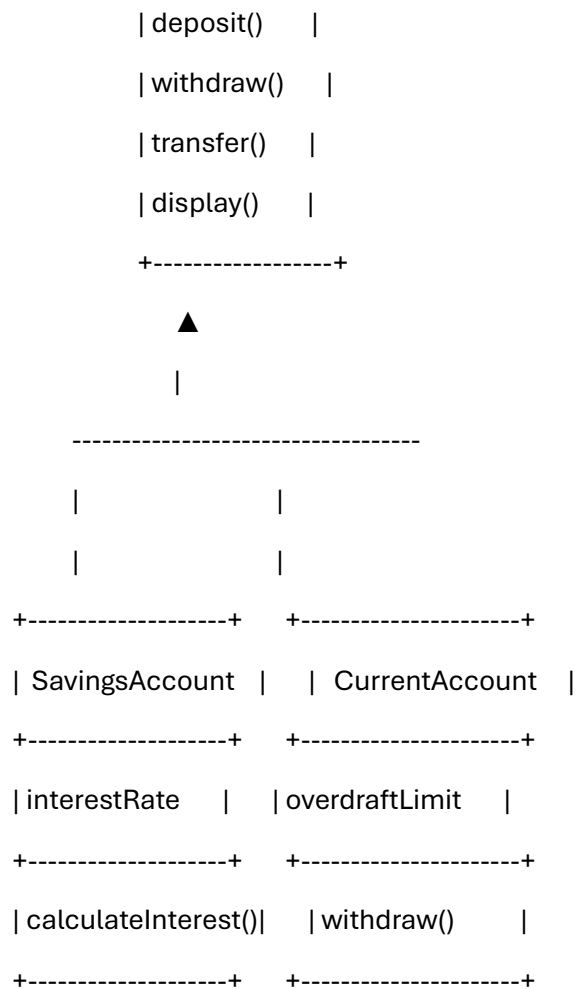
---

## 4. UML Diagram

```

+-----+
| Account |
+-----+
| accountNumber |
| balance      |
| accountType  |
+-----+

```



## 5. Java Program

```

class Account {
    String accountNumber;
    double balance;
    String accountType;

    Account(String accountNumber, double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.accountType = accountType;
    }

    void deposit(double amount) {

```

```

        balance += amount;

        System.out.println("Deposited: " + amount);
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;

            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient Balance");
        }
    }

    void transfer(Account receiver, double amount) {
        if (amount <= balance) {
            balance -= amount;
            receiver.balance += amount;

            System.out.println("Transferred: " + amount);
        } else {
            System.out.println("Transfer Failed");
        }
    }

    void display() {
        System.out.println("\nAccount Number: " + accountNumber);
        System.out.println("Account Type: " + accountType);
        System.out.println("Balance: " + balance);
    }
}

class SavingsAccount extends Account {

```

```
double interestRate;
```

```
SavingsAccount(String accountNumber,
```

```
    double balance,
```

```
    double interestRate) {
```

```
    super(accountNumber, balance, "Savings");
```

```
    this.interestRate = interestRate;
```

```
}
```

```
void calculateInterest() {
```

```
    double interest = balance * interestRate / 100;
```

```
    balance += interest;
```

```
    System.out.println("Interest Added: " + interest);
```

```
}
```

```
}
```

```
class CurrentAccount extends Account {
```

```
    double overdraftLimit;
```

```
CurrentAccount(String accountNumber,
```

```
    double balance,
```

```
    double overdraftLimit) {
```

```
    super(accountNumber, balance, "Current");
```

```
    this.overdraftLimit = overdraftLimit;
```

```
}
```

```
@Override
```

```
void withdraw(double amount) {
```

```
    if (amount <= (balance + overdraftLimit)) {
```

```
        balance -= amount;

        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("Overdraft Limit Exceeded");
    }
}
}
```

```
public class BankManagement {

    public static void main(String[] args) {

        SavingsAccount s1 =
            new SavingsAccount("S101", 10000, 5);

        CurrentAccount c1 =
            new CurrentAccount("C201", 5000, 2000);

        s1.deposit(2000);
        s1.withdraw(1000);
        s1.calculateInterest();

        s1.transfer(c1, 3000);

        s1.display();
        c1.display();
    }
}
```

---

## 6. Sample Output

Deposited: 2000.0

Withdrawn: 1000.0

Interest Added: 550.0

Transferred: 3000.0

Account Number: S101

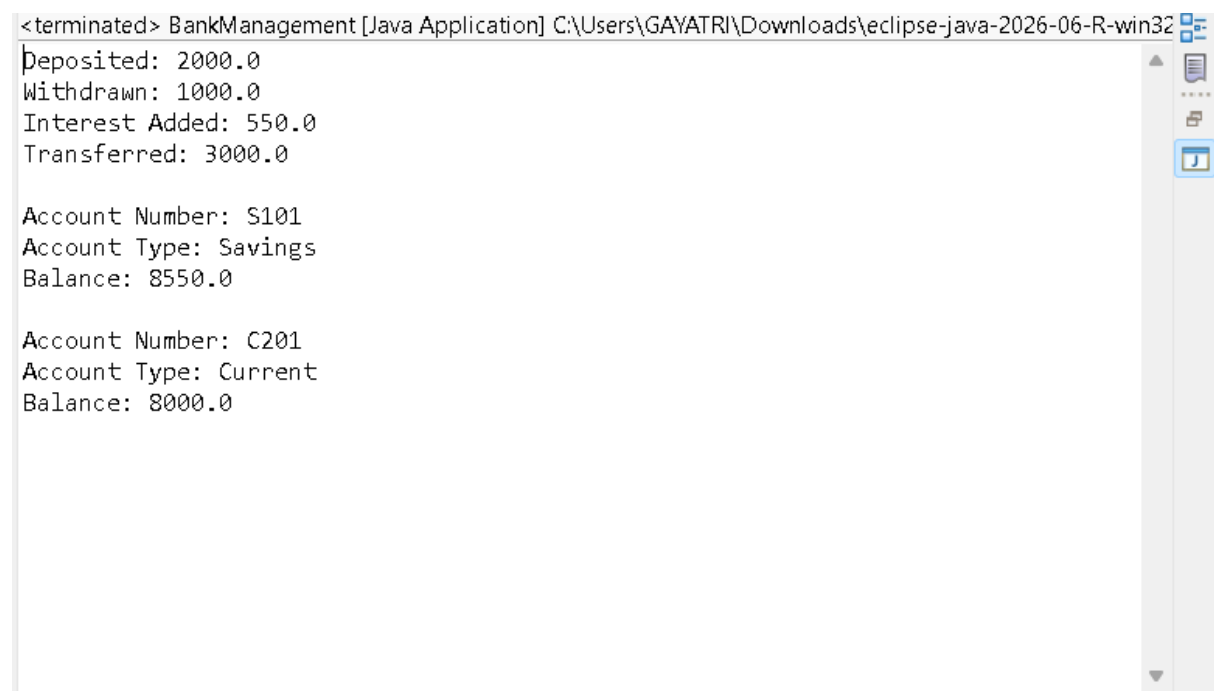
Account Type: Savings

Balance: 8550.0

Account Number: C201

Account Type: Current

Balance: 8000.0



```
<terminated> BankManagement [Java Application] C:\Users\GAYATRI\Downloads\eclipse-java-2026-06-R-win32
Deposited: 2000.0
Withdrawn: 1000.0
Interest Added: 550.0
Transferred: 3000.0

Account Number: S101
Account Type: Savings
Balance: 8550.0

Account Number: C201
Account Type: Current
Balance: 8000.0
```

## 7. Result

The **Bank Account Management System** was successfully implemented using **Classes, Objects, Inheritance, Method Overriding, and Object Interaction**. Banking operations such as deposit, withdrawal, interest calculation, and fund transfer were performed successfully.

